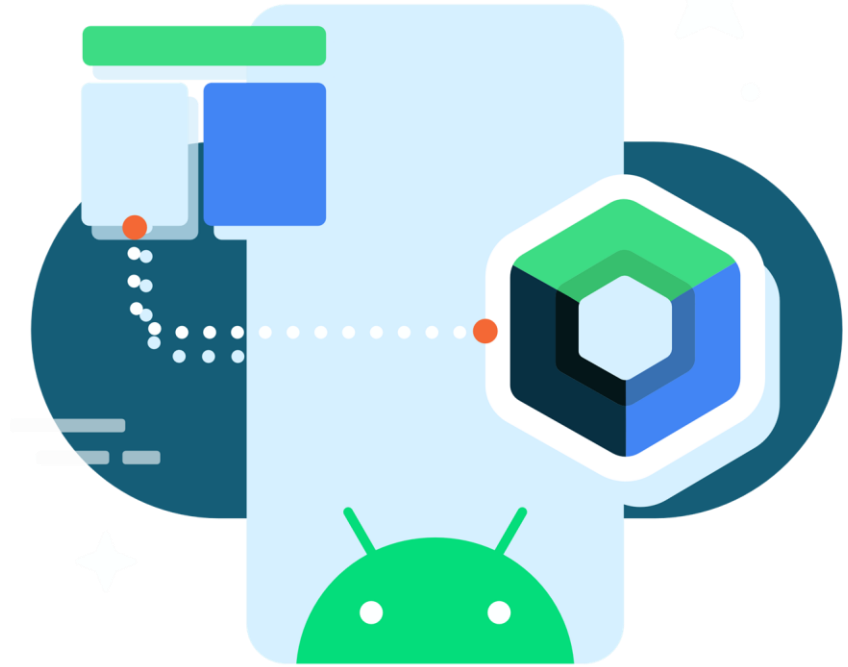
رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Mobile Application Development

Lists, Material Design, and UI interaction in Jetpack Compose



Display Lists and Use Material Design

LazyColumn & LazyRow Basics

- `LazyColumn` and `LazyRow` display lists efficiently by only rendering items visible on screen.
    - **LazyColumn**: Vertical scrolling.
    - **LazyRow**: Horizontal scrolling.

Example:

```
LazyColumn {
    items(100) { index ->
        Text("Item #$index")
    }
}
```

When to Use Lazy vs Regular Column/Row

LazyColumn & LazyRow ✅	Column & Row ⚠️
Large, dynamic data sets (unknown or large item counts)	Few static items (e.g., 3 - 4 items)
Optimized for performance	Less performance optimization
Only renders visible items	Renders all items immediately
Ideal for dynamic, scrollable lists	Ideal for fixed, small layouts

Displaying Dynamic Data in Lists

Typical Pattern:

- Data source (List from API, Database)
- Map data to UI elements (Composable)
- Display using `LazyColumn`

Example:

```
val names = listOf("Alice", "Bob", "Charlie")

LazyColumn {
    items(names) { name ->
        Text(text = name)
    }
}
```

Item Reuse & Performance

- Compose efficiently manages UI resources:
  - Recycles composables as they scroll off-screen.
  - Reduces memory usage and improves performance.

Compose handles reuse automatically.

- You just provide item definitions; Compose optimizes.

Using items and item Functions

- **items**: displays multiple items from collections.
- **item**: displays single or unique item.

```
LazyColumn {
    item {
        Text("Header")
    }
    items(100) { index ->
        Text("Item #$index")
    }
    item {
        Text("Footer")
    }
}
```

Headers, Footers & Custom List Items

```
LazyColumn {
    item { Text("Header") }
    items(listOfData) { data ->
        CustomListItem(data)
    }
    item { Text("Footer") }
}
```

Custom Item Composable:

```
@Composable
fun CustomListItem(name: String) {
    Card(
        modifier = Modifier.padding(8.dp),
        elevation = CardDefaults.cardElevation(4.dp)
    ) {
        Text(
            text = name,
            modifier = Modifier.padding(16.dp)
        )
    }
}
```

Adding Separators or Group Headers

- Use `stickyHeader` for grouped lists:

Example:

```
LazyColumn {
    stickyHeader {
        Text("Group A", modifier = Modifier.background(Color.Gray))
    }
    items(groupA) { item -> Text(item) }

    stickyHeader {
        Text("Group B", modifier = Modifier.background(Color.Gray))
    }
    items(groupB) { item -> Text(item) }
}
```

Working with Material Design Components

Jetpack Compose provides built-in Material Design components:

- **Scaffold**: Main layout structure (AppBar, FAB, Drawer)
- **Surface, Card**: Containers for UI components
- **Button, TextField, IconButton**: Interactive elements
- **MaterialTheme**: Colors, Typography, Shapes

Scaffold Example:

```
Scaffold(
    topBar = { TopAppBar(title = { Text("My App") }) },
    floatingActionButton = {
        FloatingActionButton(onClick = { /* action */ }) {
            Icon(Icons.Default.Add, "Add")
        }
    }
) {
    // Screen content here
}
```

MaterialTheme: Colors, Typography, Shapes

- Centralizes styling across your app.

```
MaterialTheme(
    colorScheme = colorScheme,
    typography = typography,
    shapes = shapes
) {
    // Compose UI
}
```

Access in Composable:

```
Text(
    text = "Hello World!",
    style = MaterialTheme.typography.headlineMedium,
    color = MaterialTheme.colorScheme.primary
)
```

Dark Mode and Theme Customization

- Detect and respond to system-wide dark mode settings.

Detecting System Theme:

```
@Composable
fun MyTheme(content: @Composable () -> Unit) {
    val darkTheme = isSystemInDarkTheme()
    val colors = if (darkTheme) darkColors else lightColors
    MaterialTheme(colorScheme = colors, content = content)
}
```

Custom Color Palette Example

Define custom colors for your theme:

```
val darkColors = darkColorScheme(
    primary = Color(0xFFBB86FC),
    secondary = Color(0xFF03DAC6)
)

val lightColors = lightColorScheme(
    primary = Color(0xFF6200EE),
    secondary = Color(0xFF03DAC6)
)
```

List Interactions & UI Feedback

Clickable Items:

```
LazyColumn {
    items(dataList) { item ->
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .clickable { /* Handle click */ }
                .padding(16.dp)
        ) {
            Text(item)
        }
    }
}
```

Swipe-to-Dismiss (Advanced Interaction)

Ideal for actions like delete or archive.

Using `SwipeToDismiss` from Compose **Material**:

```
val dismissState = rememberDismissState()

SwipeToDismiss(
    state = dismissState,
    background = { /* background UI when swiped */ },
    dismissContent = {
        Text("Swipe to remove")
    }
)
```

Swipe-to-Dismiss (Advanced Interaction)

Ideal for actions like delete or archive.

Using `SwipeToDismissBox`:

```
val dismissState = rememberSwipeToDismissBoxState()

SwipeToDismissBox(
    state = dismissState,
    backgroundContent = { /* background UI when swiped */ },
    content = {
        Text("Swipe to remove")
    }
)
```

Animations in Lists

Animate item appearance:

```
LazyColumn {
    items(itemsList, key = { it.id }) { item ->
        AnimatedVisibility(
            visible = true,
            enter = fadeIn(),
            exit = shrinkVertically() + fadeOut()
        ) {
            Text(item.name)
        }
    }
}
```

رواد مصر الرقمية

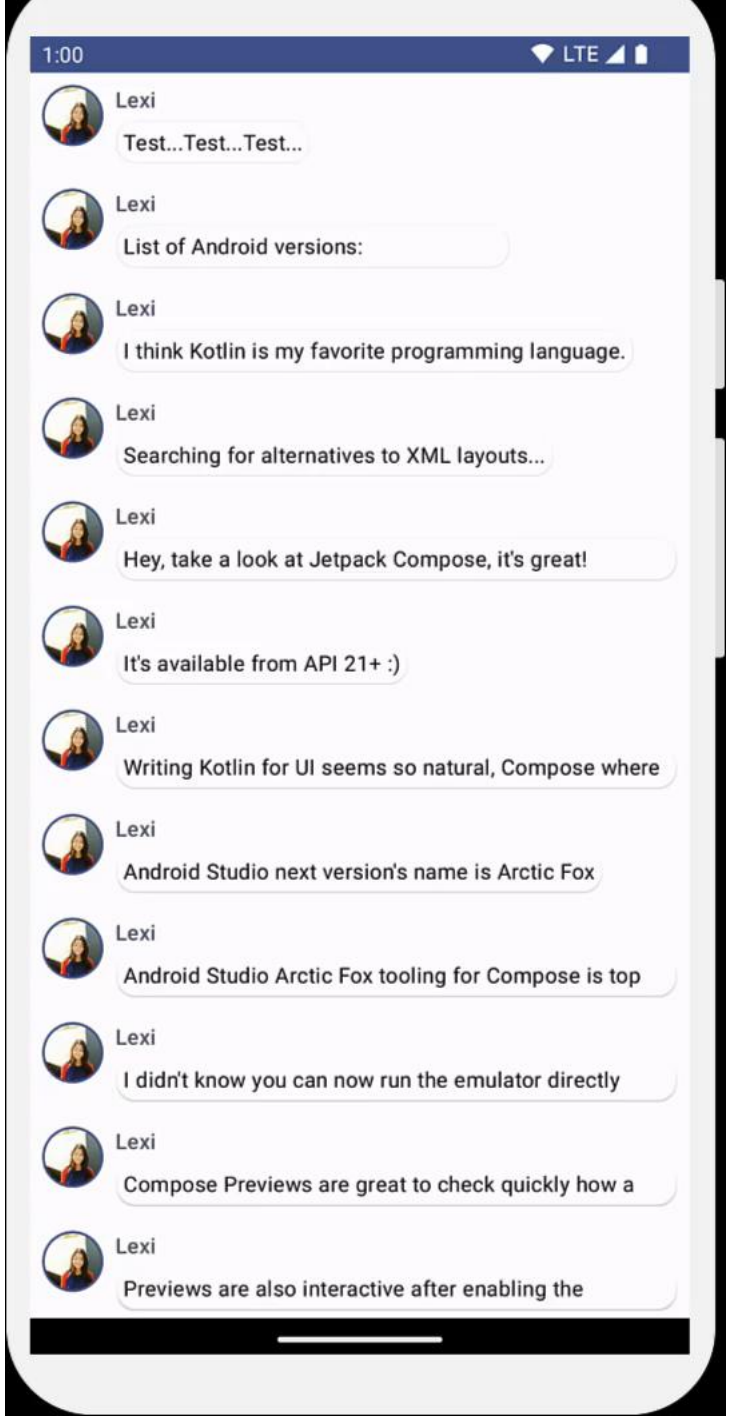
وزارة الاتصالات وتكنولوجيا المعلومات

Exercise 1

You've built a simple chat screen efficiently showing a list of expandable & animated messages containing an image and texts, designed using Material Design principles with a dark theme included and previews

Copy the sample dataset into your project to help bootstrap the conversation quickly.

Try to implement the exercise alone without any help to consolidate the information.
If you need some help, use this link
https://developer.android.com/develop/ui/compose/tutorial



Quiz Time

Key Takeaways

- Use `LazyColumn` **and** `LazyRow` for efficient scrolling lists.
- Utilize built-in **Material components** to build consistent UIs.
- Leverage `MaterialTheme` to manage colors, typography, and shapes.
- Implement responsive and accessible interactions (clickable, swipe, animations).
- Optimize list performance by allowing Compose to handle item reuse automatically.

THANK YOU